



Automating forest classification on Microsoft Fabric

Session 1 · Environment and data foundations

JDI Woodlands · Four hours · From an empty workspace to a Delta table you could query today

Session 1 of 2 · Bring a laptop, a workspace, and one real question about your own data



What you leave with today

- ▶ A Fabric workspace and a Lakehouse that you built, not one that was handed to you
- ▶ A Woodlands stand register landed as a Delta table, reprojected to the New Brunswick grid
- ▶ A decision rule for picking a tool chain that you can apply without asking anyone
- ▶ A working understanding of why bronze, silver and gold exist, rather than that they do
- ▶ One homework task that puts your own operating area into the pipeline

If you leave with only one thing, make it the layer contracts. Everything else is tooling.



Running order

TIME	BLOCK	MIN
0:00	Pre-flight, environment and access check, with a fallback for anyone blocked	20
0:20	Owning an analytics product, what we are and are not building	20
0:40	Your environment tool by tool	45
1:25	Choosing a tool chain, three problem patterns and how to pick	30
1:55	Break	15
2:10	Fabric and OneLake, workspaces, Lakehouse, shortcuts, SQL endpoint	40
2:50	Hands-on, land a dataset, reproject it, write a Delta table	50
3:40	Wrap and homework brief	20

Two checkpoints, at 1:25 and 3:25. Nobody carries a broken environment past a checkpoint.



How today works



Checkpoints

Two of them. At each one you run a validation cell that prints PASS or FAIL per assertion. Green means carry on.



Solution notebooks

Every exercise has a complete answer key. Using it at a checkpoint is the designed path, not giving up.



Fallback tiers

Shared workspace, then pairing, then read-only. Three ways to keep going, none of them a failure state.



Ask early

Five minutes stuck is learning. Twenty is waste. Put it in the Teams channel and someone answers.



One real question

Bring a question about your own data. The last block of Session 2 is where it connects.



We run to time

The hands-on blocks absorb overrun from everything before them, so the tour ends when the timer says.

BLOCK 1

0:00 · 20 min



Pre-flight

Find broken environments while there is still time to fix them



What the pre-flight checks

Standard library only, so it runs before any pip install has happened



Python and Spark

The session starts, and it starts in under a couple of minutes.



Lakehouse attached

lh_woodlands is the default. Nothing works without this.



Delta write and read

A real write and a real read, then a clean-up. Permissions are proved, not assumed.



Outbound DNS

The Spark session can resolve planetarycomputer.microsoft.com.



STAC API reachable

A live call to the Sentinel-2 collection endpoint.



Foundry endpoint

Expected to be missing today. Notebook 04 has an offline path.



Triage: symptom, cause, fix

SYMPTOM	CAUSE	FIX
No default lakehouse	Notebook opened outside the Lakehouse context	Explorer pane, add lh_woodlands, set default, restart session
Delta write denied	Workspace on a Pro licence, or you have Viewer role	Move to the shared capacity workspace
DNS resolution fails	Tenant blocks outbound traffic from Spark	Cached-scene fallback, staged in the shared workspace
STAC ok, assets 403	Signing step skipped	Every href must pass through planetary_computer.sign
Import errors	Environment not attached	Run the pip cell, or attach env-woodlands-geo
Session start over 5 min	Capacity cold or throttled	Tell a facilitator, do not retry



If you are still blocked, in this order

- 1 Shared workspace** **Tier 1**
ws-woodlands-shared, write access, every solution notebook already run. You work there and lose nothing.
- 2 Pair up** **Tier 2**
Two people, one keyboard, and the person who is blocked drives. This is the best of the three and nobody ever picks it first.
- 3 Read-only** **Tier 3**
Work locally against the committed sample outputs and rejoin at the next break.



Owning an analytics product

What separates a pipeline someone maintains from a notebook someone once ran



A notebook that ran, and a product someone owns

Product

- ✓ Has a named user making a named decision
- ✓ Refreshes on a schedule nobody has to remember
- ✓ Fails loudly, in a way that reaches a human
- ✓ Every number traces back to a source without a re-run
- ✓ A second person has changed it without asking the author

Notebook

- ✗ Has an author, and the author is the interface
- ✗ Refreshes when someone remembers to open it
- ✗ Fails silently, or produces a plausible wrong answer
- ✗ Numbers are reproducible only by re-running everything
- ✗ Exactly one person can safely change it

An analytics product with exactly one person who can run it is a risk with a dashboard attached.



Before you write a line of code, answer these



Who is the user

A Woodlands planner deciding where to send a crew next season. Not a data scientist, and not a dashboard.



What decision changes

Which stands get visited, and in what order. If no decision changes, you are building a screensaver.



How often it refreshes

Monthly. That single answer determines the imagery cadence, the compute cost and the whole architecture.



What done looks like

A number on a map, with a date attached, that a forester will act on without ringing you first.



What it must never do

Make a silvicultural decision on its own, or produce a number nobody can trace.



Who defends it

Whoever is in the room when a regulator asks. Design for that conversation now.



What we are not building today

- ▶ A research-grade classifier. Published accuracy figures need labelled ground truth we do not have
- ▶ A replacement for field cruising. This narrows where a crew goes; it does not tell you what is there
- ▶ Anything that detects change below a 20 metre pixel, so selective thinning stays invisible
- ▶ A system that makes a decision. Every output lands in front of a person who decides
- ▶ A permanent answer. Thresholds get argued with, and the argument is the point

Being explicit about the limits is what makes the claims that remain defensible.



The three contracts



BRONZE

fidelity to source

- Fidelity to what arrived
- Append only, never overwritten
- No reprojection, no masking, no renaming
- Stores unsigned URLs, never a credential



SILVER

comparability

- Comparability across scenes and dates
- Cloud masked, reflectance scaled, reprojected once
- Indices computed, pixel accounting carried
- No business rules and no thresholds



GOLD

meaning for a decision

- Meaning for a named decision
- Classification, change detection, narrative
- Every number traces to a silver row
- Star schema, ready for Direct Lake

The test: if a number is questioned, you can walk it back to a bronze row without re-running anything.

BLOCK 3

0:40 · 45 min



Your environment, tool by tool

Five tools, one live action each. Nobody watches a slide about a product they have not touched



Five tools, five live actions

TOOL	TIME	WHAT YOU DO, RIGHT NOW
Microsoft Fabric	12 min	Create a workspace, create lh_woodlands, look at Files and Tables
Power BI	8 min	Open the SQL analytics endpoint, run one SELECT, see the same data
Planetary Computer	10 min	Search Sentinel-2 for your area, read one STAC item properly
GitHub Copilot	10 min	Ask for a reprojection snippet, then find the mistake in the answer
Microsoft Foundry	5 min	Open a project, find a deployed model, note the endpoint and version

The Copilot segment is deliberately adversarial. You are looking for the bug, not the answer.



Microsoft Fabric

- ▶ One SaaS surface over one storage account, so Spark, T-SQL and Power BI read the same bytes
- ▶ The workspace is the unit of access, capacity and lifecycle. Arguments about permissions are arguments about workspaces
- ▶ Capacity is what you are billed for, and throttling is a capacity event, not a workspace one
- ▶ Items live in a workspace: Lakehouse, Notebook, Pipeline, Semantic model, Report
- ▶ Your action: create ws-woodlands-yourname, create lh_woodlands, and open both Files and Tables



Power BI and the SQL analytics endpoint

- ▶ Every Lakehouse ships with a read-only T-SQL endpoint over the same Delta files
- ▶ No copy, no load step, no second refresh schedule. A different engine on identical bytes
- ▶ This is the fastest way to sanity check a table you just wrote from a notebook
- ▶ Metadata sync is asynchronous, so a table can take a minute or two to appear
- ▶ Your action: open the endpoint, run one SELECT against a table, and notice you loaded nothing

Mention the sync lag now. It looks like a bug at 3:10 and costs ten minutes of debugging.



Planetary Computer and STAC

- ▶ A catalogue of open Earth observation data, with Sentinel-2 Level 2A as our source
- ▶ STAC is the query standard: Catalog, then Collection, then Item, then Asset
- ▶ You search by geometry, date and property, and get back links to Cloud Optimized GeoTIFFs
- ▶ Assets need signing, and the signature expires, so you sign at the point of use
- ▶ Your action: open the Explorer, draw a box on your area, and read one item's properties



GitHub Copilot, used adversarially

- ▶ It writes most of the code in Session 2, and that is a good outcome, not a compromise
- ▶ It optimises for a plausible answer to the question you asked
- ▶ Geospatial work is full of questions people ask imprecisely, which is where it goes wrong
- ▶ The skill being taught is reading the answer, not generating it
- ▶ Your action: ask it to reproject a GeoDataFrame to New Brunswick coordinates, then find the bug

Run the same prompt twice. The two answers differ, and that difference is the lesson.



The same task, two prompts

```
# Prompt A: "reproject this GeoDataFrame to New Brunswick coordinates"
gdf = gdf.to_crs("EPSG:2953")          # raises if gdf.crs is None
gdf["area"] = gdf.geometry.area       # what unit? nobody said

# Prompt B: "the frame is in WGS84 but has no CRS set. Declare it, reproject
#           to EPSG:2953, then compute area in hectares."
gdf = gdf.set_crs(4326)                # declare what it already is
gdf = gdf.to_crs(2953)                # then transform
gdf["area_ha"] = gdf.geometry.area / 10_000
```

Same model, same session, thirty seconds apart. The difference is the constraints in the prompt, not the intelligence of the tool.



Microsoft Foundry

- ▶ Where the language model lives, as a named, versioned deployment behind an endpoint
- ▶ In this pipeline it does exactly one job: turning numbers into a sentence a planner reads
- ▶ It never classifies, never calculates, and never decides. Session 2 shows why
- ▶ The deployment name is recorded alongside every generated sentence, so wording changes are explainable
- ▶ Your action: find the project, find a deployment, and write down its name and version

No Foundry access today? Everything still runs. Notebook 04 has an offline path that produces the same table.

BLOCK 4

1:25 · 30 min



Choosing a tool chain

Three problem patterns, and the honest criteria for picking between them



Ask three questions, and stop at the first clear answer

1

Is the data tabular with a known schema

If yes, take the lowest-code option that handles it. You will maintain whatever you choose.

Question 1

2

Does it need per-file, per-scene or per-geometry work

If yes, you need code, which means a Spark notebook. Nothing else can open the file.

Question 2

3

Does it need judgement over unstructured content

If yes, and only if yes, a language model belongs in the chain.

Question 3



Three patterns and their defaults

PATTERN	REACH FOR	DO NOT REACH FOR
Tabular, known schema, scheduled refresh	Dataflow Gen2 to land it, T-SQL to transform	A Spark notebook, because you own it forever
Raster or vector geospatial, per-scene processing	Spark notebook with GeoPandas, rioxarray and rasterio	Dataflows, which have no raster story at all
Unstructured text or images, judgement calls	A Foundry model behind a notebook or function	A trained classifier you must label data for this quarter

There is a fourth option that wins more often than any of these: no new tool, because a lookup already answers it.



Exercise: pick a chain and defend it

Two minutes at your table, one minute to defend. One of these is a trap.



Card 1 · Scaling data

40,000 rows a month from the scaling contractor. Same columns every month. Join to the block register, refresh by the third.



Card 2 · Canopy closure

Which planted stands from the last five years show lower than expected canopy closure, from imagery, quarterly, full licence area.



Card 3 · Field notes

12,000 free-text field notes. Each mentions a block id somewhere. Link them to the register so a supervisor can browse by block.



Card 4 · Tooltip summary

A two-sentence plain-language condition summary per stand, from the twelve numeric columns the pipeline already produces.



“I reach for a notebook when the data will not open any other way.

I reach for a model when the output is language and the numbers come from somewhere I already trust. Otherwise I reach for the lowest-code option a colleague can maintain without me.”

Write your own version of this before the break, and keep it.

BLOCK 5

1:55 · 15 min



Break

Fifteen minutes. Facilitators are unblocking the pre-flight failures

BLOCK 6

2:10 · 40 min

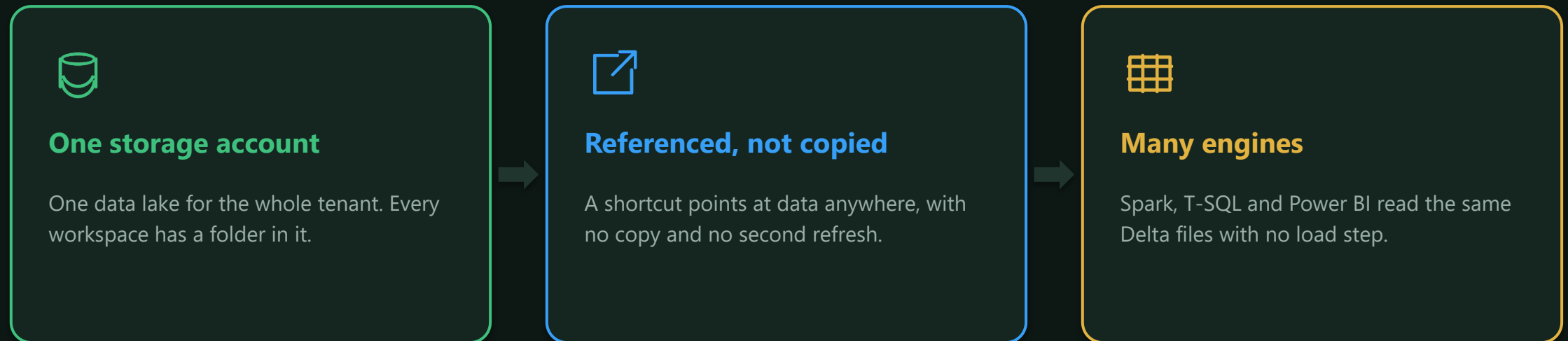


Fabric and OneLake

Workspaces, Lakehouse, shortcuts, and the SQL analytics endpoint



OneLake in one picture



Most of the copying in a typical estate exists to move data between engines. This removes the reason for it.



Anatomy of a Lakehouse



Files/

Anything at all. GeoTIFFs, JSON, a zip somebody sent you. Bronze rasters live here.



Tables/

Delta only. A table is a folder of Parquet plus a transaction log, which is where atomicity and time travel come from.



Shortcuts

A reference to data elsewhere, in another workspace or another cloud. Permissions follow the source, not the shortcut.



SQL analytics endpoint

Read-only T-SQL over the same files. No storage, no load, no separate model to keep in sync.



Time travel

The transaction log lets you read the table as it was. This is what makes bronze auditable.



Schemas

Optional grouping inside Tables. Worth turning on if your tenant offers it, awkward to add later.



Live demo: shortcut, then query through T-SQL

```
-- 1. Create a shortcut from lh_woodlands to the shared reference Lakehouse
-- Tables > New shortcut > OneLake > lh_reference > stand_register
-- No copy. No refresh schedule. Nothing moves.
```

```
-- 2. Then, in the SQL analytics endpoint, with no load step at all:
```

```
SELECT species_group,
        COUNT(*)           AS stands,
        ROUND(SUM(area_ha), 1) AS total_ha
FROM    stand_register
GROUP BY species_group
ORDER BY total_ha DESC;
```

The same bytes, written by Spark, referenced by a shortcut, read by T-SQL. This is usually the moment the OneLake idea lands.



Where each layer lives

LAYER	LOCATION	WRITE MODE	WHY
Bronze	Files/bronze/ and Tables/bronze_*	Append	Overwriting destroys the ability to explain last month
Silver	Tables/silver_*	Merge on grain	Re-running a period must replace it, not duplicate it
Gold	Tables/gold_*	Overwrite per period	Derived and cheap to rebuild from silver
Model	Tables/gold_dim_* and gold_stand_facts	Overwrite	Star schema, shaped for Direct Lake

Bronze append-only is the line that pays for itself. A failed run costs time and nothing else.

BLOCK 7

2:50 · 50 min

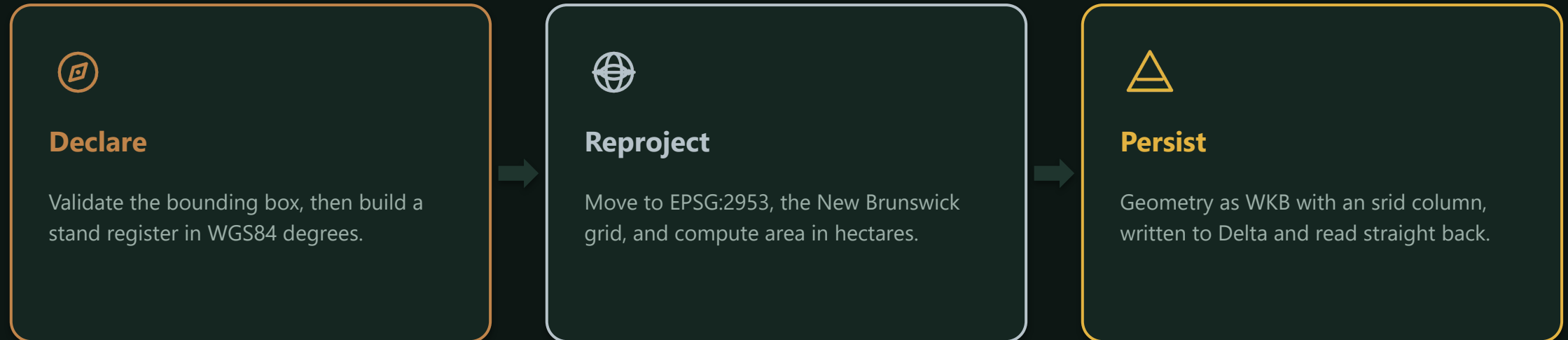


Hands-on

Land a Woodlands dataset, reproject it, write a Delta table



What you are about to build



Fifty minutes. The validation cells tell you where you are, so you never have to guess.



The three traps, in the order they appear

TRAP	WHAT YOU SEE	WHY IT HAPPENS
geometry column to Delta	AnalysisException: cannot resolve 'geometry'	Spark has no geometry type. Serialise to WKB, carry the srid separately
to_crs on a naive frame	ValueError: cannot transform naive geometries	GeoPandas refuses to guess, which is correct. Use set_crs to declare, then to_crs
Area before reprojection	Areas like 0.0004	Square degrees. Looks plausible until someone checks it against a cruise sheet
set_crs on projected data	No error at all, geometry in the wrong hemisphere	set_crs relabels without moving anything. This is the dangerous one

Only the last one is silent. Print `gdf.crs` and `gdf.total_bounds` before and after every transform.



The checkpoint at 3:25

[PASS] register is not empty	120 stands
[PASS] CRS is the analysis projection	EPSG:2953
[PASS] stand_id is unique	
[PASS] all expected columns present	
[PASS] areas are plausible	18.4 to 41.2 ha
[PASS] natural stands have no planting year	
[PASS] row count survives the round trip	120 vs 120
[PASS] areas match after the round trip	max drift 0.00e+00 ha

Every FAIL is a specific, named thing. Open the solution for that step, read it, carry on. Nobody sits on a red line while the room moves.

BLOCK 8

3:40 · 20 min



Wrap and homework

What we covered, and what to do before Session 2



What we covered

- ▶ Bronze keeps faith with what arrived, silver makes things comparable, gold means something to someone
- ▶ One storage account, many engines, and a shortcut instead of a copy
- ▶ Area and distance are only meaningful in a projected coordinate system, and `set_crs` is not `to_crs`
- ▶ Geometry travels through Delta as WKB with the srid in its own column, always
- ▶ The tool chain question is answered in three steps, and you stop at the first clear answer

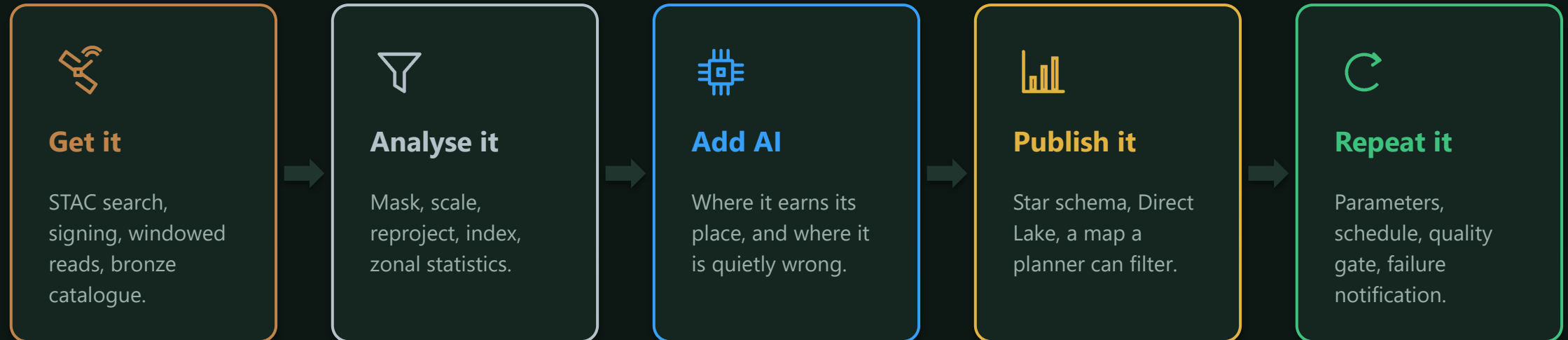


Homework, about 45 minutes

- 1 Choose your area of interest** **15 min**
A block you know, roughly 10 to 40 km on a side. Get the bounding box from the Planetary Computer Explorer.
- 2 Check imagery availability** **15 min**
Sentinel-2 L2A, last growing season, 20 percent cloud or less. You want at least three scenes over your box.
- 3 Land your stand register** **15 min**
Re-run notebook 00 with your bounding box. Then check the areas against stands you actually know.



Session 2, in five steps



Bring your bounding box, your scene count, and one real question about your own data.



Before you leave

- ▶ Confirm your workspace name and Lakehouse are saved somewhere you will find them
- ▶ Note your bounding box, or plan the fifteen minutes to find one
- ▶ Post any unresolved blocker in the Teams channel today, not on the morning of Session 2
- ▶ Bring the one question about your own data. It is the most useful thing in your bag